Dive into Deep Learning (D2L)

Chapter 2: Preliminaries

Joon Soo Yoo, Ph.D.

Spring 2026

Assistant Professor
Defense & Advanced Security (DAS) Lab
Jeonbuk National University (JBNU)

2.1.4 Broadcasting: 형상이 다른 텐서의 연산

- **Broadcasting**: 원소별 연산을 수행할 때, 특정 조건 하에 자동으로 모양을 맞춰주는 메커니즘

2단계 동작 원리

1. **확장(Expand)**: 하나 혹은 두 배열의 원소를 복사하여 축의 길이를 일치시킴
2. **연산(Operation)**: 일치된 모양을 바탕으로 원소별 연산 수행

Python Code: (3x1) + (1x2) → (3x2)

```
a = torch.arange(3).reshape((3, 1)) # [[0], [1], [2]]
b = torch.arange(2).reshape((1, 2)) # [[0, 1]]
print(a + b)
```

2.1.5 Saving Memory: 메모리 절약과 id 확인

- `Y = X + Y`와 같은 연산은 새로운 메모리를 할당하고 `Y`가 그곳을 가리키게 함
- 수억 개의 파라미터를 다루는 딥러닝에서는 **메모리 낭비**와 **참조 오류**의 원인이 됨

메모리 주소 변경 확인 (id)

```
before = id(Y)
Y = Y + X
print(id(Y) == before) # False (새로운 메모리 할당됨)
```

- **In-place 연산**: 기존에 할당된 메모리 공간의 값만 직접 수정하여 주소를 유지함

2.1.5 Saving Memory: In-place 연산 구현

- 슬라이스 표기법([:])이나 복합 대입 연산자(+=)를 사용

Python Code: 주소를 유지하는 방법

```python
# 방법 1: 슬라이스 전체 지정
Z = torch.zeros_like(Y)
Z[:] = X + Y  # Z의 주소는 변하지 않음

# 방법 2: 복합 대입 연산자 사용 (가장 권장됨)
before = id(X)
X += Y
print(id(X) == before) # True
```

- **장점**: 메모리 오버헤드를 줄이고 시스템 안정성을 높임

2.1.6 Conversion: NumPy 및 스칼라 변환

- PyTorch 텐서와 NumPy 배열은 **메모리를 공유**함 (한쪽 수정 시 다른 쪽도 변경)

주요 변환 함수

함수	용도
`X.numpy()`	텐서를 NumPy 배열로 변환
`torch.from_numpy(A)`	NumPy 배열을 텐서로 변환
`x.item()`	크기가 1인 텐서를 파이썬 스칼라 값으로 변환

Python Code: Scalar 추출

```
a = torch.tensor([3.5])
print(a.item(), float(a), int(a)) # 3.5, 3.5, 3
```

2.2 Data Preprocessing

데이터 사전처리

2.2.1 데이터 준비: Kaggle 데이터셋 다운로드

- 실전 환경과 유사하게 외부 데이터 저장소(Kaggle)에서 데이터를 직접 가져옴
- **kagglehub**: 캐글 데이터셋을 코드로 간편하게 내려받는 라이브러리

Python Code: 데이터 다운로드

```python
import kagglehub

# 데이터셋의 최신 버전을 다운로드하고 경로(path)를 반환받음
path = kagglehub.dataset_download("mengting098/house-tiny")

print("Path to dataset files:", path)
```

경로 이해하기

다운로드된 경로는 시스템 내부의 임시 폴더(`/root/.cache/...`)에 저장됨. 우리는 이 주소가 담긴 `path` 변수를 활용해 파일에 접근함.

2.2.2 Data Preparation: 데이터 분리

- 지도 학습(Supervised Learning)을 위해 데이터를 두 부분으로 나눔
- **Inputs**: 예측을 위한 특징값(Features)
- **Targets**: 예측하고자 하는 정답값(Labels)

Python Code: iloc을 이용한 분리

```
# 모든 행(:)에 대해 0~1열은 inputs, 2열은 targets로 설정
inputs, targets = data.iloc[:, 0:2], data.iloc[:, 2]
```

Missing Values (결측값)

CSV의 비어있는 값(NA)은 Pandas에서 NaN으로 표시됨. 데이터 과학의 "빈대(Bed bugs)"와 같아서 반드시 처리가 필요함.

2.2.2 결측값 처리: 범주형 (Categorical)

- **Imputation (대체)**: 범주형 데이터의 NaN을 하나의 카테고리로 간주
- **One-hot Encoding**: 문자열 데이터를 숫자로 변환

Python Code: get_dummies 활용

```
# dummy_na=True: NaN 값도 하나의 열로 생성함
inputs = pd.get_dummies(inputs, dummy_na=True)
print(inputs)
```

변환 결과 예시 (RoofType)

Original	RoofType_Slate	RoofType_nan
Slate	1 (True)	0 (False)
NaN	0 (False)	1 (True)

2.2.2 One-hot Encoding: 왜 열을 쪼개는가?

- **개념**: 범주형 데이터를 각 카테고리에 해당하는 n개의 이진(Binary) 열로 확장하는 방식

범주형 데이터 처리의 두 방식

구분	수치 라벨링 (0, 1, 2...)	원-핫 인코딩 (0 or 1)
방식	하나의 열에 숫자를 부여	각 항목마다 새로운 열 생성
위험성	모델이 데이터 간의 **서열/순서**를 오해함	데이터가 늘어나지만 독립성이 보장됨
결론	순서가 있는 데이터에만 권장	대부분의 범주형 데이터에 표준으로 사용

Python Code: pd.get_dummies()의 결과

```
# 'RoofType' 컬럼에 Slate와 NaN이 있는 경우
# Slate인 행: [RoofType_Slate=1, RoofType_nan=0]
# NaN인 행:   [RoofType_Slate=0, RoofType_nan=1]
```

- **핵심**: 컴퓨터가 "사과(1) + 배(2) = 포도(3)"와 같은 잘못된 수식 관계를 학습하지 않도록 방지함

2.2.2 결측값 처리: 수치형 (Numerical)

- 수치 데이터의 NaN은 해당 열의 **평균값(Mean)**으로 채우는 것이 일반적임

Python Code: fillna 활용

```
# NumRooms의 비어있는 값을 평균값으로 대체
inputs = inputs.fillna(inputs.mean())
print(inputs)
```

계산 로직

- 현재 NumRooms 값: 2.0, 4.0 (다른 두 행은 NaN)
- 평균 계산: $(2.0 + 4.0)/2 = 3.0$
- 결과: 모든 NaN 자리가 **3.0**으로 채워짐

2.2.3 Conversion: Pandas에서 Tensor로

- 모든 데이터가 수치화(Numerical)되었으므로, 이제 모델 학습을 위해 텐서로 변환함
- **흐름**: Pandas DataFrame → NumPy Array → PyTorch Tensor

Python Code: Tensor 변환

```
import torch

# 1. inputs(특징)와 targets(정답)를 텐서로 변환
# to_numpy()를 통해 넘파이 구조를 거쳐 텐서로 생성
X = torch.tensor(inputs.to_numpy(dtype=float))
y = torch.tensor(targets.to_numpy(dtype=float))

print("Features (X):\n", X)
print("\nLabels (y):\n", y)
```

Output 확인

```
(tensor([[3., 0., 1.], [2., 0., 1.], [4., 1., 0.], [3., 0., 1.]],
        dtype=torch.float64),
 tensor([127500., 106000., 178100., 140000.], dtype=torch.float64))
```

Section 2.2 요약: 데이터 전처리 흐름

- 딥러닝 모델 학습을 위한 데이터 준비 4단계

Data Pipeline Summary

단계	주요 작업
1. Load	`pd.read_csv()`로 로우 데이터 불러오기
2. Split	`iloc`으로 입력(X)과 정답(y) 분리
3. Clean	결측값(NaN) 처리 (평균 대체, 원-핫 인코딩 등)
4. Convert	`torch.tensor()`로 최종 변환

메모

데이터의 타입이 기본적으로 `float64`로 생성될 수 있음. 메모리 효율을 위해 나중에 `float32`로 필요시 변환.

2.3 Linear Algebra

선형대수학 기초

2.3.1 Scalars: 하나의 값을 가진 데이터

- **Scalar**: 단 하나의 숫자만 포함하는 값 (x, y, z 등 소문자로 표기)
- **수학적 표기**: $x \in \mathbb{R}$ (실수 집합 $\mathbb{R}$에 속하는 스칼라 x)

Python Code: 스칼라 연산

```
import torch


x = torch.tensor(3.0)
y = torch.tensor(2.0)

# 사칙연산 및 거듭제곱
print(x + y, x * y, x / y, x**y)
```

- 파이토치에서 스칼라는 **원소가 1개인 텐서**로 구현됨

2.3.2 Vectors: 스칼라의 고정 길이 배열

- **Vector**: 스칼라 값들의 나열 ($\mathbf{x}, \mathbf{y}$ 등 굵은 소문자로 표기)
- **수학적 표기**: $\mathbf{x} \in \mathbb{R}^n$ (n개의 요소를 가진 벡터)
- **시각화**: 기본적으로 요소를 세로로 쌓은 **열 벡터(Column Vector)**로 간주

Python Code: 벡터 생성 및 인덱싱

```
x = torch.arange(3) # [0, 1, 2] 생성
print(x)

# 인덱싱 (Zero-based indexing)
print(x[2]) # 출력: tensor(2)
```

2.3.2 벡터의 속성: 길이와 모양

- **Length**: 벡터에 포함된 요소의 총 개수 (`len(x)`)
- **Shape**: 각 축(Axis)의 길이를 나타내는 튜플 (`x.shape`)

용어의 혼동 주의 (Overloading "Dimension")

딥러닝에서는 '차원'이라는 단어가 두 가지 의미로 섞여 쓰임:

1. **Order (차수)**: 축(Axis)의 개수 (예: 벡터는 1차 텐서)
2. **Dimensionality (차원성)**: 특정 축을 따라 있는 요소의 개수 (예: n차원 벡터)

Check Point

`x = torch.arange(3)` 일 때:

- `len(x)`: 3
- `x.shape`: `torch.Size([3])`

2.3.2 심화: Order(차수) vs Dimensionality(차원성)

- **문제**: 딥러닝에서는 "차원"이라는 단어를 서로 다른 두 의미로 혼용함

1. Order (Rank/Axis)

"데이터 구조의 깊이"

- 축(Axis)이 몇 개인가?
- 대괄호([ ])가 몇 겹인가?
- `x.ndim`으로 확인
- *예: 벡터는 항상 1차 텐서*

2. Dimensionality

"축에 담긴 요소의 개수"

- 특정 축의 길이는 얼마인가?
- 성분(Feature)이 몇 개인가?
- `x.shape`으로 확인
- *예: 성분이 5개면 5차원 벡터*

아파트 비유 (The Apartment Analogy)

- **Order**: 건물의 **층수** (1층 건물인가, 2층 건물인가?)
- **Dimensionality**: 한 층에 있는 **방의 개수** (방이 3개인가, 500개인가?)

2.3.2 실습: ndim과 shape의 차이

Python Code: 차원 확인

```
import torch
# 5개의 성분을 가진 벡터 생성
x = torch.tensor([1, 2, 3, 4, 5])

print(x.ndim)   # Order 확인 -> 결과: 1 (축이 1개인 벡터)
print(x.shape)  # Dimensionality 확인 -> 결과: torch.Size([5])
```

- **결론**: 위 텐서는 **"1차(Order) 텐서"**이면서 **"5차원(Dimensionality) 벡터"**임

2.3.3 Matrices: 데이터의 표(Table) 표현

- **Matrix**: 2개의 축(Axis)을 가진 2차(2nd-order) 텐서
- **수학적 표기**: $\mathbf{A} \in \mathbb{R}^{m \times n}$ (굵은 대문자로 표기)
  - m: 행(Row)의 개수, n: 열(Column)의 개수
  - $m = n$인 경우, **정방 행렬(Square Matrix)**이라 부름

Python Code: 행렬 생성

```
# 0부터 5까지의 숫자로 3x2 행렬 생성
A = torch.arange(6).reshape(3, 2)
print(A)
```

Output

```
tensor([[0, 1],
        [2, 3],
        [4, 5]])
```

2.3.3 Transpose: 행과 열의 뒤바꿈

- **전치(Transpose)**: 행렬의 주대각선을 기준으로 행과 열을 교환함
- **수학적 정의**: $\mathbf{B} = \mathbf{A}^\top$ 일 때, $b_{ij} = a_{ji}$

Python Code: A.T

```
# 3x2 행렬 A를 전치하여 2x3 행렬로 변환
print(A.T)
```

Output

```
tensor([[0, 2, 4],
        [1, 3, 5]])
```

2.3.3 Symmetric Matrices: 거울 같은 행렬

- **대칭 행렬**: 전치를 해도 원래 행렬과 동일한 정방 행렬
- **조건**: $\mathbf{A} = \mathbf{A}^\top$

Python Code: 대칭성 확인

```
A = torch.tensor([[1, 2, 3], [2, 0, 4], [3, 4, 5]])
print(A == A.T)
```

특징

주대각선을 기준으로 위쪽(Upper triangle)과 아래쪽(Lower triangle)이 거울처럼 대칭을 이룸

2.3.3 Matrices in ML: 실전 데이터의 구조

- 머신러닝에서 행렬은 데이터셋을 표현하는 표준 도구임

데이터셋 행렬 (X)

구분	의미
행 (Rows)	각각의 개별 데이터 샘플 (Records)
열 (Columns)	데이터의 고유 속성 (Features / Attributes)

예시: 집값 예측 데이터

- **행**: 집 1, 집 2, 집 3...
- **열**: 방 개수, 지붕 타입, 가격...

2.3.4 Tensors: 고차원 데이터의 일반화

- **Tensor**: 스칼라, 벡터, 행렬을 포함하여 임의의 축(Axis) 개수를 가진 다차원 배열

Python Code: 3차원 텐서 생성 (2x3x4)

```
# 0~23까지의 숫자로 (2, 3, 4) 모양의 텐서 생성
X = torch.arange(24).reshape(2, 3, 4)
print(X)
```

실전 활용: 이미지 데이터

- **3차원 텐서**: [Height, Width, Channel(RGB)] → 단일 이미지
- **4차원 텐서**: [Batch, Height, Width, Channel] → 이미지 묶음

2.3.4 High-order Tensors: Batch의 의미

- **Batch (배치)**: 연산의 효율성을 위해 묶어서 처리하는 데이터의 개수

4차원 텐서의 구조: [B H W C]

- **B (Batch)**: 이미지의 개수 (예: 32장, 64장...)
- **H (Height)**: 이미지의 세로 픽셀 수
- **W (Width)**: 이미지의 가로 픽셀 수
- **C (Channel)**: 색상 채널 (RGB = 3)

Batch

이미지 한 장을 처리하더라도 딥러닝 모델은 보통 4차원 입력을 기대함. 이때는 `[1, 224, 224, 3]` 과 같이 배치 크기를 1로 설정하여 전달함.

※ GPU는 병렬 처리에 최적화되어 있으므로, 배치를 적절히 설정하는 것이 학습 속도에 결정적임

2.3.5 Basic Properties: 텐서 산술 연산

- **Shape 유지**: 동일한 모양의 두 텐서에 대한 원소별 연산은 결과 텐서의 모양을 변형시키지 않음

Python Code: 원소별 덧셈

```
import torch
A = torch.arange(6, dtype=torch.float32).reshape(2, 3)
B = A.clone()  # 새로운 메모리 할당하여 복사

# A와 B는 모두 (2, 3) 모양
print(A + B)
```

Output

```
tensor([[ 0., 2., 4.], [ 6., 8., 10.]])
```

2.3.5 Basic Properties: 아다마르 곱

- **Hadamard Product**: 두 행렬의 대응하는 원소끼리 곱하는 연산
- **기호**: $\mathbf{A} \odot \mathbf{B}$

수학적 정의

$$\mathbf{A} \odot \mathbf{B} = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} & \dots & a_{1n}b_{1n} \\ a_{21}b_{21} & a_{22}b_{22} & \dots & a_{2n}b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}b_{m1} & a_{m2}b_{m2} & \dots & a_{mn}b_{mn} \end{bmatrix}$$

Python Code: A * B

```
# 파이토치에서 '*' 연산자는 아다마르 곱을 수행함
print(A * B)
```

2.3.5 스칼라와 텐서의 연산

- 텐서에 스칼라를 더하거나 곱하면, 모든 원소에 해당 연산이 적용됨

Python Code: Scalar Addition & Multiplication

```
a = 2
X = torch.arange(24).reshape(2, 3, 4)

# 모든 원소에 2를 더함
print(a + X)
# 연산 후에도 Shape는 (2, 3, 4)로 동일
print((a * X).shape)
```

핵심 요약

사칙연산(+, -, *, /)은 텐서의 구조를 깨뜨리지 않고 내부 값들만 변형함

2.3.6 Reduction: 데이터의 요약

- **Reduction**: 텐서의 원소들을 결합하여 더 낮은 차원의 텐서로 만드는 작업
- **합계 (Summation)**: 가장 기본적인 리덕션 연산

수학적 정의

- 벡터의 합: $\sum_{i=1}^{d} x_i$
- 행렬의 합: $\sum_{i=1}^{m} \sum_{j=1}^{n} a_{ij}$

Python Code: 전체 합산

```
import torch
A = torch.arange(6, dtype=torch.float32).reshape(2, 3)

print(A)       # 2x3 행렬
print(A.sum()) # 모든 원소의 합 -> 결과: tensor(15.)
```

2.3.6 Reduction: 축(Axis) 지정 합계

- 특정 축을 지정하면 해당 축을 "압착"하여 결과를 생성함

axis$\bar{0}$ (행 압착)

위에서 아래로 더함 (결과는 열의 개수만큼 남음)

```
# Shape: (2, 3) -> (3,)
print(A.sum(axis=0))
```

axis$\bar{1}$ (열 압착)

왼쪽에서 오른쪽으로 더함 (결과는 행의 개수만큼 남음)

```
# Shape: (2, 3) -> (2,)
print(A.sum(axis=1))
```

- **Tip**: `axis=[0, 1]`과 같이 여러 축을 동시에 지정할 수도 있음

2.3.6 Mean: 데이터의 평균

- **Mean (Average)**: 합계를 원소의 총 개수(`numel`)로 나눈 값
- 수학적 정의: $\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$

Python Code: 평균 구하기

```
# 전체 평균
print(A.mean())

# 특정 축 기준 평균 (열 방향 평균)
print(A.mean(axis=0))
```

주의사항

`mean()` 연산은 데이터 타입이 **실수형(Float)**일 때만 가능함. 정수형 텐서인 경우 `float()` 변환 후 사용해야 함.

2.3.7 Non-Reduction Sum: keepdims=True

- **문제**: 일반적인 `sum()`은 지정한 축을 완전히 없애버림 (차원 축소)
- **해결**: `keepdims=True`를 사용하면 축의 길이를 1로 유지하며 차원 수를 보존함

Python Code: 차원 유지하며 더하기

```
# axis=1(열)을 따라 더하되 차원 유지
sum_A = A.sum(axis=1, keepdims=True)

print(sum_A)       # 결과가 (2, 1) 행렬로 유지됨
print(sum_A.shape) # torch.Size([2, 1])
```

왜 유용한가?

결과값이 여전히 2차원(2, 1)이므로, 원래의 2차원 행렬 $A(2,3)$와 **Broadcasting** 연산이 즉시 가능함

2.3.7 활용: 행별 합계가 1이 되도록 만들기

- keepdims로 구한 합계로 원래 행렬을 나누면 각 행의 합이 1이 됨

Python Code: Broadcasting Normalization

```
# A: (2, 3), sum_A: (2, 1)
# Broadcasting에 의해 각 행이 자신의 합계로 나누어짐
normalized_A = A / sum_A

print(normalized_A)
print(normalized_A.sum(axis=1)) # 각 행의 합이 정확히 1.0이 됨
```

2.3.8 Dot Products: 벡터 사이의 스칼라 합

- **Dot Product**: 같은 위치의 원소끼리 곱한 후 그 결과들을 모두 합산함
- **수학적 정의**: $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ 일 때, $x \cdot y = \sum_{i=1}^{d} x_i y_i$

Python Code: 내적 구하기

```
import torch
x = torch.arange(3, dtype=torch.float32)
y = torch.ones(3, dtype=torch.float32)

# 방식 1: 전용 함수 사용
print(torch.dot(x, y))

# 방식 2: 원소별 곱셈 후 합계 (수학적 정의와 동일)
print(torch.sum(x * y))
```

2.3.9 Matrix-Vector Products: 차원의 변환

- **개념**: 행렬 $\mathbf{A} \in \mathbb{R}^{m \times n}$와 벡터 $\mathbf{x} \in \mathbb{R}^n$의 곱은 길이 m인 새로운 벡터를 생성함
- **핵심**: 결과 벡터의 i번째 원소는 행렬 $\mathbf{A}$의 i번째 행과 벡터 $\mathbf{x}$의 내적임

수학적 구조

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} \mathbf{a}_1^\top \\ \mathbf{a}_2^\top \\ \vdots \\ \mathbf{a}_m^\top \end{bmatrix} \mathbf{x} = \begin{bmatrix} \mathbf{a}_1^\top \mathbf{x} \\ \mathbf{a}_2^\top \mathbf{x} \\ \vdots \\ \mathbf{a}_m^\top \mathbf{x} \end{bmatrix}$$

2.3.9 Matrix-Vector: 코딩 및 딥러닝 활용

- **제약 사항**: **A**의 열 개수(axis 1)와 **x**의 길이(dimension)가 일치해야 함

Python Code: mv() 및 @ 연산자

```python
# A: (2, 3) 행렬, x: (3,) 벡터
A = torch.arange(6, dtype=torch.float32).reshape(2, 3)
x = torch.arange(3, dtype=torch.float32)

# 방식 1: torch.mv() 함수
print(torch.mv(A, x))

# 방식 2: @ 연산자 (Matrix-Vector, Matrix-Matrix 모두 지원)
print(A @ x) # 결과 Shape: (2,)
```

딥러닝에서의 의미

인공신경망의 한 층(Layer)에 입력 데이터(**x**)가 들어왔을 때, 가중치 행렬(**W**)과 곱해져 다음 층으로 신호를 전달하는 과정 그 자체임

2.3.10 Matrix-Matrix Multiplication: 행렬 곱셈

- **개념**: $\mathbf{A}(m \times k)$와 $\mathbf{B}(k \times n)$의 곱은 $\mathbf{C}(m \times n)$ 행렬을 만듦
- **연산 방식**: 결과 행렬의 원소 c_{ij}는 $\mathbf{A}$의 i번째 **행**과 $\mathbf{B}$의 j번째 **열**의 **내적**

수학적 정의

$$\mathbf{C} = \mathbf{AB}, \quad c_{ij} = \mathbf{a}_i^\top \mathbf{b}_j$$

여기서 $\mathbf{a}_i^\top$는 $\mathbf{A}$의 i행, $\mathbf{b}_j$는 $\mathbf{B}$의 j열을 의미함

Matrix Multiplication vs Hadamard Product

- **Hadamard ($*$)**: 같은 위치끼리 곱 (Shape 유지)
- **Matrix (@)**: 행과 열의 결합 (Shape 변환)

2.3.10 실습: Shape 매칭 이해하기

- 행렬 곱셈이 가능하려면 **앞의 열(k)**과 **뒤의 행(k)** 숫자가 반드시 일치해야 함

Python Code: torch.mm() 및 @

```
import torch
A = torch.arange(6, dtype=torch.float32).reshape(2, 3) # (2, 3)
B = torch.ones(3, 4)                                   # (3, 4)

# 두 방식 모두 행렬 곱셈 수행
C1 = torch.mm(A, B)
C2 = A @ B

print(C1.shape) # 결과: torch.Size([2, 4])
print(C1)
```

Output

```
tensor([[ 3., 3., 3., 3.], [12., 12., 12., 12.]])
```

2.3.11 Norms: 벡터의 '크기' 측정

- **Norm**: 벡터를 스칼라값으로 매핑하여 "크기"를 나타내는 함수 $f(\mathbf{x})$
- **노름의 3가지 조건**:
  1. **Scaling**: $\|\alpha\mathbf{x}\| = |\alpha|\|\mathbf{x}\|$
  2. **Triangle Inequality**: $\|\mathbf{x}+\mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$
  3. **Non-negativity**: $\|\mathbf{x}\| \geq 0$ (0일 때만 $\|\mathbf{x}\| = 0$)

핵심 개념

노름은 벡터의 요소 개수(Dimensionality)가 아니라, 요소들이 가진 **값의 물리적 크기**를 측정함

2.3.11 L2 Norm vs L1 Norm

L_2 Norm (Euclidean)

피타고라스 정리 기반의 직선 거리

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$$

```
u = torch.tensor([3.0, -4.0])
torch.norm(u) # 결과: 5.0
```

L_1 Norm (Manhattan)

절대값의 합 (격자 무늬 거리)

$$\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i|$$

```
torch.abs(u).sum() # 결과: 7.0
```

2.3.11 Frobenius Norm: 행렬 전용 노름

- **Frobenius Norm**: 행렬을 하나의 긴 벡터로 간주하고 L_2 노름을 계산한 것과 같음
- **정의**: 행렬의 모든 원소의 제곱의 합에 루트를 씌움

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} a_{ij}^2}$$

Python Code: Frobenius Norm

```
# 4x9 행렬(모든 원소 1)의 노름 계산
# 1을 36번 더하고 루트 -> sqrt(36) = 6
print(torch.norm(torch.ones((4, 9))))
```

2.3.11 Why Norms? 딥러닝에서의 목적

- 딥러닝은 결국 **최적화(Optimization)** 문제임

노름의 실제 활용

활용 분야	노름의 역할
Loss Function	예측값과 실제값 사이의 **거리(오차)**를 측정함
Regularization	가중치 행렬의 노름을 제한하여 **과적합(Overfitting)** 방지
Similarity	두 데이터 표현 사이의 거리를 계산하여 유사도 측정

"우리가 '거리를 줄인다' 혹은 '오차를 최소화한다'고 말할 때, 수학적으로는 항상 어떤 노름을 최소화하고 있는 것이다."

2.3 Summary (1): 선형대수의 기본 객체

- 딥러닝 데이터는 축(Axis)의 개수에 따라 다음과 같이 분류됨

Tensors at a Glance

객체	Axis 수	표기법	의미
Scalar	0	x	단일 숫자
Vector	1	$\mathbf{x}$	숫자의 나열 (특징 벡터)
Matrix	2	$\mathbf{A}$	데이터 테이블 (샘플 $\times$ 특징)
Tensor	n	$\mathbf{X}$	다차원 배열 (이미지, 비디오 등)

Key Takeaway

Dimension은 축의 개수(Order)와 축의 길이(Dimensionality)라는 두 의미로 쓰이므로 문맥에 따라 구분할 것!

2.3 Summary (2): 연산의 구분

- 텐서 연산은 크게 두 부류로 나뉨

원소별 연산 (Hadamard)

- 기호: $\mathbf{A} \odot \mathbf{B}$ (`A * B`)
- 모양이 유지됨
- 계산 복잡도: 이차 시간 (Quadratic)

행렬 곱셈 (Matrix Product)

- 기호: $\mathbf{AB}$ (`A @ B`)
- 모양이 변환됨 (축소/확장)
- 계산 복잡도: **삼차 시간 (Cubic)**
  $\rightarrow$ 훨씬 무거운 연산!

- **Reduction**: `sum`, `mean` 등을 통해 특정 축을 따라 데이터를 요약할 수 있음

2.3 Summary (3): Norms and Distance

- **Norm**은 데이터의 크기나 두 데이터 사이의 거리를 측정함

주요 노름 요약

종류	이름	용도
L_2 Norm	Euclidean	일반적인 직선 거리 측정
L_1 Norm	Manhattan	이상치(Outlier)에 덜 민감한 거리 측정
Frobenius	-	행렬의 전체 에너지를 측정

Final Message

딥러닝의 목표는 결국 **손실 함수(Loss function)인 노름을 최소화**하는 최적의 가중치 행렬을 찾는 과정임

2.4 Calculus

미분: 최적화를 위한 엔진

2.4.1 Derivatives: 변화율의 수학적 정의

- **미분(Derivative)**: 입력값의 아주 미세한 변화에 따른 출력값의 변화 비율
- **수학적 정의**: 함수 f의 점 x에서의 미분 $f'(x)$

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

기하학적 의미

함수 그래프 위의 특정 지점에서의 **접선의 기울기(Slope)**를 의미함. 이 기울기를 따라가면 함수의 값이 어디로 증가/감소하는지 알 수 있음.

2.4.1 실습: 극한(Limit)의 수치적 확인

- $f(x) = 3x^2 - 4x$ 일 때, $x = 1$에서의 미분값 확인 ($f'(1) = 2$)

Python Code: Numerical Limit

```
def f(x):
    return 3 * x**2 - 4 * x

h = 0.1
for _ in range(5):
    # (f(1+h) - f(1)) / h 계산
    numerical_limit = (f(1+h) - f(1)) / h
    print(f'h={h:.5f}, limit={numerical_limit:.5f}')
    h *= 0.1 # h를 0에 가깝게 줄임
```

- **결과**: h가 작아질수록 값은 이론적인 미분값인 **2.0**에 수렴함

2.4.1 주요 미분 공식과 연산 법칙

Common Derivatives

- $\frac{d}{dx}x^n = nx^{n-1}$
- $\frac{d}{dx}e^x = e^x$
- $\frac{d}{dx}\ln(x) = \frac{1}{x}$
- $\frac{d}{dx}\sin(x) = \cos(x)$

Rules of Calculus

- **Sum**: $(f+g)' = f' + g'$
- **Product**: $(fg)' = f'g + fg'$
- **Quotient**: $(\frac{f}{g})' = \frac{f'g - fg'}{g^2}$

딥러닝에서의 활용

우리가 최소화하려는 **손실 함수(Loss Function)**도 수많은 미분 가능한 함수들의 조합으로 이루어져 있으며, 위 법칙들을 적용해 기울기를 계산함.

2.4.3 Partial Derivatives: 편미분

- **개념**: 변수가 여러 개인 함수($y = f(x_1, x_2, \ldots, x_n)$)에서 관심 있는 하나의 변수 x_i에 대해서만 변화율을 측정
- **방법**: x_i를 제외한 나머지 모든 변수를 **상수(Constant)** 취급함

수학적 정의 및 표기법

$$\frac{\partial y}{\partial x_i} = \lim_{h \to 0} \frac{f(\ldots, x_i + h, \ldots) - f(\ldots, x_i, \ldots)}{h}$$

$$\frac{\partial y}{\partial x_i} = \frac{\partial f}{\partial x_i} = \partial_{x_i} f = f_{x_i} = D_i f$$

핵심 원리

"다른 것들은 다 멈춰있다고 가정하고, 딱 x_i 하나만 움직였을 때 함수가 어떻게 변하니?"라고 묻는 것임

2.4.3 Gradients: 기울기의 집합체

- **Gradient**: 모든 변수에 대한 편미분 값을 한데 모아놓은 **벡터**
- **수학적 표기**: $\nabla_{\mathbf{x}} f(\mathbf{x})$ (나블라 f 라고 읽음)

정의

함수 $f : \mathbb{R}^n \to \mathbb{R}$ 에 대하여:

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = [\partial_{x_1} f(\mathbf{x}), \partial_{x_2} f(\mathbf{x}), \ldots, \partial_{x_n} f(\mathbf{x})]^\top$$

기하학적 의미

그레이디언트 벡터는 함수 값이 **가장 가파르게 증가하는 방향**을 가리킴. 학습 시에는 이 반대 방향($-\nabla f$)으로 이동하여 손실을 최소화함.

2.4.3 Matrix Calculus: 벡터/행렬 미분 법칙

- 복잡한 신경망 수식을 하나하나 편미분하는 대신 사용하는 공식들

주요 행렬 미분 공식

함수 $f(\mathbf{x})$	Gradient $\nabla_{\mathbf{x}} f(\mathbf{x})$
$\mathbf{A}\mathbf{x}$	$\mathbf{A}^\top$
$\mathbf{x}^\top\mathbf{A}$	$\mathbf{A}$
$\mathbf{x}^\top\mathbf{A}\mathbf{x}$	$(\mathbf{A}+\mathbf{A}^\top)\mathbf{x}$
$\|\mathbf{x}\|^2$	$2\mathbf{x}$
$\|\mathbf{X}\|_F^2$ (Frobenius)	$2\mathbf{X}$

학습 포인트

기본적인 미분 법칙($x^2 \to 2x$)이 벡터와 행렬로 확장되어도 그 형태가 매우 유사함에 주목할 것!

2.4.3 증명: $\nabla_{\mathbf{x}}\mathbf{A}\mathbf{x} = \mathbf{A}^\top$

- **목표**: $\mathbf{y} = \mathbf{A}\mathbf{x}$ 일 때, $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$의 결과가 왜 $\mathbf{A}^\top$인지 증명

Step 1: 성분별 분해

$\mathbf{y} = \mathbf{A}\mathbf{x}$의 i번째 성분 y_i는 다음과 같음:

$$y_i = \sum_{j=1}^{n} A_{ij} x_j = A_{i1}x_1 + A_{i2}x_2 + \cdots + A_{in}x_n$$

Step 2: 편미분 수행

y_i를 x_k에 대해 편미분하면, x_k 앞에 붙은 계수만 남음:

$$\frac{\partial y_i}{\partial x_k} = A_{ik}$$

2.4.3 증명: $\nabla_{\mathbf{x}}\mathbf{A}\mathbf{x} = \mathbf{A}^\top$

Step 3: 행렬 재구성

모든 i, k에 대해 이 미분값들을 모으면 행렬 $\mathbf{A}$가 됨.

$$\nabla_{\mathbf{x}}(\mathbf{A}\mathbf{x}) = \left(\frac{\partial \mathbf{A}\mathbf{x}}{\partial \mathbf{x}}\right)^\top = \mathbf{A}^\top$$

2.4.4 Chain Rule: 연쇄 법칙의 기초

- **배경**: 딥러닝 모델은 수많은 함수가 겹겹이 쌓인 거대한 합성함수임
- **연쇄 법칙**: 합성함수 $y = f(g(x))$의 미분은 각 단계 미분의 곱으로 표현됨

일변수 연쇄 법칙 (Single Variable)

$y = f(u)$ 이고 $u = g(x)$ 일 때:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

3.1 Linear Regression

선형 회귀: 수치 예측의 기초

3.1 Basics: 머신러닝의 언어

- **Regression (회귀)**: 주식, 온도, 집값처럼 **연속적인 수치**를 예측하는 문제

데이터셋의 구성 요소

용어	설명
Training Set	모델을 학습시키기 위한 데이터의 집합
Example / Sample	데이터셋의 한 줄 (Row), 개별 데이터 포인트
Feature (특징)	예측을 위해 사용되는 입력 변수 (예: 면적, 방 개수)
Label / Target	우리가 맞추고자 하는 정답 (예: 실제 매매가)

Notation

- n: 샘플의 개수, d: 특징(Feature)의 개수
- $\mathbf{x}^{(i)}$: i번째 샘플, $x_j^{(i)}$: i번째 샘플의 j번째 특징

3.1.1 Linear Model: 선형 관계의 가정

- **핵심 가정**: "정답(y)은 입력(x)들의 가중치 합에 편향을 더한 형태일 것이다!"

실제 예시: 집값 예측

$$\text{price} = w_{\text{area}} \cdot \text{area} + w_{\text{age}} \cdot \text{age} + b$$

- w (Weights): 가중치 (각 특징이 결과에 주는 영향력)
- b (Bias): 편향 (모든 입력이 0일 때의 기본값)

모델의 목표

우리가 가진 데이터를 가장 잘 설명하는 최적의 **가중치(w)**와 **편향(b)**을 찾는 것

3.1.1 수식의 효율화: 벡터와 행렬

- 일일이 더하는 대신 선형대수학의 **내적**과 **행렬 곱**으로 깔끔하게 표현함

벡터 표현 (샘플 1개)

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b$$

- $\mathbf{w}, \mathbf{x} \in \mathbb{R}^d$
- 점(Point) 예측에 사용

행렬 표현 (Batch 처리)

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$$

- $\mathbf{X} \in \mathbb{R}^{n \times d}$ (Design Matrix)
- 한 번에 모든 샘플 예측

※ 여기서 b를 더할 때는 이전 장에서 배운 **Broadcasting**이 자동으로 적용됨

3.1.1.2 Loss Function: 모델의 '부적합도' 측정

- **Loss Function**: 모델의 예측이 실제 데이터와 얼마나 일치하지 않는지 수치화
- 값이 작을수록 모델이 데이터를 잘 설명하고 있음을 의미함

Squared Error (제곱 오차)

개별 샘플 i에 대한 오차 $l^{(i)}$는 다음과 같이 정의함:

$$l^{(i)}(\mathbf{w}, b) = \frac{1}{2}\left(\hat{y}^{(i)} - y^{(i)}\right)^2$$

- **상수** $1/2$: 미분했을 때 지수 2와 상쇄되어 수식을 깔끔하게 만들기 위한 도구
- **제곱의 의미**: 오차가 커질수록 손실값이 기하급수적으로 커져 모델이 큰 오차를 피하도록 유도함

3.1.1.2 Total Loss: 훈련 세트 전체의 오차

- 개별 샘플의 오차를 모두 더해 평균을 낸 것을 **훈련 손실(Training Loss)**이라 함

Total Loss (평균 제곱 오차 MSE)

$$L(\mathbf{w}, b) = \frac{1}{n}\sum_{i=1}^{n} l^{(i)}(\mathbf{w}, b) = \frac{1}{n}\sum_{i=1}^{n} \frac{1}{2}\left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)}\right)^2$$

데이터 포인트와 모델의 거리

기하학적으로는 직선(모델)과 각 점(데이터) 사이의 수직 거리의 제곱을 최소화하는 과정임

3.1.1.2 최적화의 수학적 목표

- 머신러닝 학습의 최종 목적은 전체 손실 $L(\mathbf{w}, b)$를 최소화하는 파라미터를 찾는 것

Objective Function

$$\mathbf{w}^*, b^* = \underset{\mathbf{w}, b}{\operatorname{argmin}}\, L(\mathbf{w}, b)$$

- $\mathbf{w}^*, b^*$: 모델이 찾아야 할 최적의 정답 가중치와 편향
- **특징**: 제곱 오차 함수는 **볼록 함수(Convex Function)**이므로, 적절한 방법을 쓰면 반드시 전역 최적해(Global Minimum)를 찾을 수 있음

3.1.1.4 Minibatch SGD

미니배치 확률적 경사 하강법

3.1.1.4 반복적 최적화의 필요성

- **문제**: 대부분의 딥러닝 모델은 수학적으로 한 번에 정답(Analytical solution)을 구하기 어려움
- **해결**: **경사 하강법(Gradient Descent)**을 통해 손실 함수를 점진적으로 줄여나가는 반복적인 파라미터 업데이트를 수행함

핵심 아이디어

현재 위치에서 손실 함수의 기울기(Gradient)가 가장 가파른 방향의 **반대 방향**으로 조금씩 이동하여 최저점에 도달함

3.1.1.4 효율적인 학습을 위한 전략 비교

GD vs SGD vs Minibatch SGD

방식	데이터 사용량	특징
Batch GD	전체 데이터셋	안정적이지만 매우 느림
SGD	샘플 1개	빠르지만 업데이트가 매우 불안정함
Minibatch SGD	**데이터의 일부 (묶음)**	**속도와 안정성의 균형 (표준)**

- **Minibatch의 이점**: 현대의 GPU 하드웨어는 여러 데이터를 한꺼번에 병렬 처리하는 데 최적화되어 있어 연산 효율이 극대화됨

3.1.1.4 Minibatch SGD의 작동 원리

1. 모델 파라미터 $(\mathbf{w}, b)$를 무작위로 초기화함
2. 훈련 데이터에서 무작위로 미니배치 $\mathcal{B}_t$를 샘플링함
3. 해당 배치의 평균 손실에 대한 기울기를 계산하여 업데이트함

파라미터 업데이트 수식

$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}_t} \partial_{(\mathbf{w}, b)} l^{(i)}(\mathbf{w}, b)$$

- η: 학습률(Learning Rate), $|\mathcal{B}|$: 배치 크기(Batch Size)

3.1.1.4 업데이트의 방향: "오르막의 반대편으로"

- **Gradient (∇L)**의 수학적 의미: 함수 값이 **가장 가파르게 증가**하는 방향
- **우리의 목표**: 손실(L)을 **최소화** (가장 낮은 바닥으로 이동)

공식의 첫 단추

바닥으로 내려가려면 기울기가 가리키는 방향의 **반대 방향**으로 움직여야 함

$$\text{방향} = -\nabla L(\mathbf{w}, b)$$

- 이 "마이너스" 기호가 우리가 산을 내려가고 있다는 수학적 증거임

3.1.1.4 보폭과 업데이트: "얼마나 멀리 갈 것인가?"

- 방향을 정했다면, 이제 **보폭(Step size)**을 결정해야 함

업데이트 공식의 구성

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \nabla L(\mathbf{w}, b)$$

- η **(Learning Rate)**: 학습률. 한 걸음의 크기를 결정하는 상수
  - 너무 크면 바닥을 지나쳐 버림 (Overshooting)
  - 너무 작으면 바닥까지 가는데 평생 걸림 (Slow convergence)

3.1.1.4 최종 공식: 미니배치의 지혜

- 전체 데이터의 기울기를 구하는 대신, 일부(Minibatch)의 **평균 기울기**를 사용

최종 업데이트 수식

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}_t} \nabla_{\mathbf{w}} l^{(i)}(\mathbf{w}, b)$$

수식 기호의 의미 총정리

기호	역할
$-$	내리막길로 가겠다는 방향 설정
η	보폭 조절 (Hyperparameter)
$\frac{1}{\lvert\mathcal{B}\rvert} \sum$	미니배치 샘플들의 의견을 종합하여 **평균**을 냄
$\nabla l^{(i)}$	개별 샘플이 가리키는 "정답으로 가는 길"

3.1.1.4 하이퍼파라미터 (Hyperparameters)

- **하이퍼파라미터**: 모델이 스스로 학습하지 못하고 사용자가 직접 설정해야 하는 값들
  - **Learning Rate (η)**: 한 걸음의 크기. 너무 크면 발산하고, 너무 작으면 느림
  - **Batch Size**: 보통 32에서 256 사이의 2의 거듭제곱 값을 선호함

모델 성능 평가

학습이 끝난 후에는 학습에 쓰이지 않은 별도의 **검증 데이터셋(Validation dataset)**을 통해 모델의 실제 품질을 확인해야 함

3.1.1.5 Predictions: 학습 후의 활용

- **Prediction**: 학습된 파라미터($\mathbf{w}, b$)를 사용하여 새로운 데이터에 대한 결과를 도출하는 단계

$$\hat{y} = \hat{\mathbf{w}}^\top \mathbf{x} + \hat{b}$$

용어의 혼동 주의: Prediction vs Inference

- **Deep Learning**: 보통 이 단계를 **Inference(추론)**라고 부름
- **Statistics**: Inference는 파라미터의 특성을 추론하는 통계적 과정을 의미함
- **주의**: 통계학자와 대화할 때는 용어 선택에 유의할 것 (본 교재는 **Prediction**을 선호함)

3.1.2 Vectorization: 속도를 위한 필수 기술

- **문제**: 파이썬의 기본 for-loop는 인터프리터 언어 특성상 연산 속도가 매우 느림
- **해결**: 벡터화(Vectorization)를 통해 선형대수 라이브러리(C++/CUDA 최적화)를 직접 호출

실험 설계: 10 000차원 벡터 덧셈

1. **방법 A**: `for-loop`로 하나씩 더하기
2. **방법 B**: 파이토치 중첩 연산자(+)로 한 번에 더하기

질의 & 응답

Any Question?